A PROJECT REPORT ON

Mar Thoma H.S. School

Session 2026-27 · Computer Science / Informatics Practices

Shram Setu 2.0

A Verified Digital Work and Wage Record for India's Informal Workforce

Tamper-evident work pass · Geofenced muster · Statutory wage protection · Cryptographic audit trail



SUBMITTED BY

Name: _____

Class: _____ Roll No.: _____

GUIDED BY

Teacher: _____

Department: _____

Submitted in partial fulfilment of the science and innovation exhibition requirement of the school.

DECLARATION

Declaration

I, _____, a student of **Mar Thoma H.S. School**, hereby declare that the project entitled **“Shram Setu 2.0 — A Verified Digital Work and Wage Record for Informal Labour”** has been designed and developed by me during the academic session Session 2026-27, under the guidance of my subject teacher.

The ideas, source code, interface design, wage arithmetic, cryptographic audit logic and documentation presented in this report are my own work. Wherever I have referred to statutory wage notifications, government welfare schemes or published technical standards, the source has been acknowledged in the References section. I further declare that this project has not been submitted, either in part or in full, for any other examination or exhibition.

I understand that Shram Setu 2.0 is an educational prototype. It does not claim to be an official government portal, and it does not perform identity verification on behalf of any authority. Every limitation of the prototype has been stated openly in Chapter 16 of this report, so that no reader mistakes a demonstration for a statutory service.

_____ _____

Signature of the Student

Name: _____
Class: _____ Roll No.: _____

Signature of the Guide

Subject Teacher, Mar Thoma H.S. School
Date: _____

Place: Indore, Madhya Pradesh

TO WHOMSOEVER IT MAY CONCERN

Certificate

This is to certify that the project work entitled **“Shram Setu 2.0 — A Verified Digital Work and Wage Record for Informal Labour”** has been carried out by _____, a student of **Class _____** of **Mar Thoma H.S. School**, during the academic session Session 2026-27.

The project has been examined and found to be satisfactory in respect of design, technical execution, documentation and presentation. The student has demonstrated a clear understanding of the problem of informal employment, of statutory wage arithmetic, and of the responsible handling of personal data. The work is recommended for the school innovation exhibition.

During the course of the work the student independently implemented a bilingual public website, two role-based portals, an offline single-file edition, an integer paise wage engine, a hash-chained audit trail and a 75-assertion automated test bench. The results reported in this document were re-run and verified in the presence of the undersigned.

_____ _____

Subject Teacher / Guide

Principal

Mar Thoma H.S. School
Indore, Madhya Pradesh

School Seal
Date: _____

ACKNOWLEDGEMENT

Acknowledgement

I record my sincere gratitude to the Principal and the faculty of **Mar Thoma H.S. School** for encouraging an original piece of work rather than a borrowed one. My subject teacher guided every stage of this project: the framing of the problem, the discipline of testing, and the honesty required when a feature did not work as intended.

I am grateful to the labour welfare and construction professionals who explained, in plain language, how daily wages are recorded on an Indian construction site — how a muster roll is maintained, how an advance is adjusted against a week of work, and why a worker who cannot read a ledger often has no way to prove what is owed. That conversation shaped the entire design of this system. The interface was built for a person standing in the sun, holding a borrowed phone, not for a person sitting at a desk.

I also acknowledge the open documentation of the e-Shram registry, the Building and Other Construction Workers (Regulation of Employment and Conditions of Service) Act, 1976, the Madhya Pradesh Minimum Wages schedule, and the Web Crypto and Web Application Manifest standards, all of which informed the technical decisions recorded in this report.

Finally, I thank my family for tolerating months of evening debugging, and my classmates who tested early builds and reported, with some relish, every button that refused to respond. Several of the defects listed in Chapter 16 were found by them.

_____

Name and Signature of the Student

Mar Thoma H.S. School, Indore · Session 2026-27

CONTENTS

Table of Contents

Declaration 2

Certificate 3

Acknowledgement 4

Table of Contents 5

Abstract — Executive Summary 8

The eight decisions that matter 8

What was measured 9

How this report is organised 9

01 · Introduction and Motivation 10

1.1 A notebook is not a record 10

1.2 Objectives 10

1.3 Scope of this submission 10

1.4 Method 11

1.5 A note on honesty in a school project 12

02 · Understanding the Problem 13

2.1 Four failures, not one 13

2.2 What a worker actually carries 13

2.3 Why not simply use a bank account? 14

2.4 Statutory anchors 14

03 · System Architecture 15

3.1 Shape of the system 15

3.2 Module inventory 15

3.3 Routing 16

3.4 Performance strategy 16

04 · Design Language and User Experience 17

4.1 Designing for a person in the sun 17

4.2 Motion, with an exit door 18

4.3 Accessibility as a feature, not a checkbox 18

4.4 A type specimen 19

05 · The Public Website 20

5.1 The verification page 20

5.2 An honest 404 20

5.3 Language on the public shell 21

06 · The Worker Portal 22

6.1 The punch, and the duplicate it caused 22

6.2 Speaking the ledger aloud 22

6.3 An account that starts empty 23

6.4 What the worker never sees 23

07 · The Contractor Dashboard 24

7.1 The scanner, and why it is only half the answer 24

7.2 Approval, not assertion 24

7.3 A number nobody has to trust 25

08 · The Smart Shram Pass and Verification 26

8.1 What the QR code contains 26

8.2 Printing, laminating and carrying 26

8.3 What verification establishes 27

8.3 Printing, laminating and carrying 27

09 · The Wage Engine 29

9.1 Why integers 29

9.2 The breakdown, in the order it is computed 29

9.3 Worked example 30

10 · The Cryptographic Audit Trail 31

10.1 Canonical serialisation 31

10.2 Four verification outcomes 31

10.3 A fragment of a real chain 32

11 · Deterministic Anomaly Detection 33

11.1 The most instructive rule 33

11.2 How the benchmark rule speaks 33

11.3 Anatomy of an alert 34

12 · Accounts, Roles and Data Protection 35

12.1 Lifecycle 35

12.2 Passwords 35

12.3 Where the data lives 35

12.4 The exhibition mode 36

13 · Bilingual Access and Language 37

14 · The Offline Single-File Edition . . . 38

14.1 Four defects found by building it . . . 38

14.2 Storage when storage is denied . . . 38

14.3 What works offline, and what does not . . . 39

15 · Testing and Validation . . . 40

15.1 The fourteen unit groups . . . 40

15.2 The flow test, step by step . . . 41

15.3 Defects found and fixed . . . 41

16 · Results, Limitations and Honest Disclosures . . . 42

16.1 Results . . . 42

16.2 Limitations, stated plainly . . . 42

16.3 A five-minute demonstration . . . 43

17 · Future Scope . . . 44

17.1 A server, and a conflict rule . . . 44

17.2 Independent witnesses . . . 44

17.3 Institutional integration . . . 44

17.4 Field hardening . . . 44

17.5 A plausible next-term plan . . . 45

18 · Conclusion . . . 46

References . . . 47

Appendix A · Route Reference . . . 48

Deep-link grammar . . . 48

Appendix B · Test Index . . . 49

Offline harness . . . 49

Rendered screens . . . 49

Appendix C · Glossary . . . 50

Appendix D · Selected Source Excerpts . . . 51

D.1 Integer paise conversion . . . 51

D.2 Canonical serialisation . . . 51

D.3 The balance rule . . . 51

ABSTRACT

Executive Summary

India employs hundreds of millions of people in informal, daily-wage and construction work. For most of them the record of a day's labour exists only in a supervisor's notebook, a contractor's memory or a thumb impression on a muster roll. When a dispute arises the worker has no artefact to point at, and the employer has only a claim of dishonesty to answer it. The result is not usually fraud; it is opacity, and opacity always costs more to the weaker party.

Shram Setu 2.0, built as this project's submission for the **Mar Thoma H.S. School** science and innovation exhibition (Session 2026-27), is a working software system that gives an informal worker a small but durable artefact: a digital work pass, a day-by-day muster record, and a wage ledger whose arithmetic the worker can audit. Eight design decisions carry most of that weight.

The eight decisions that matter

1. **A pass with no personal data.** The QR code on a worker's pass encodes only an opaque reference token and a verification address. It contains no Aadhaar number, no wage figure, no photograph of an identity document — so photographing the card reveals nothing about the person holding it.
2. **One authoritative wage engine.** Money is computed in integer paise, never floating-point, and only one function is permitted to decide a balance. Payments reduce the balance; they can never increase earnings. The logic is verified by a unit test suite.
3. **Attendance that cannot be counted twice.** A second punch for the same worker on the same calendar date is merged rather than credited again, and the matching ledger row is revised instead of duplicated.
4. **A hash-chained audit trail.** Every consequential event — a punch, an approval, a payment, a dispute — is appended to a chain whose integrity can be recomputed. Editing history in place breaks the chain visibly.
5. **Anomaly rules that accuse no one.** Seven deterministic checks surface patterns that deserve a human conversation, and each states what it examined and what it cannot conclude.
6. **Two portals, one vocabulary.** The worker and the contractor see the same event from their own side of the counter, in both English and Hindi, using the same 297-key bilingual dictionary.
7. **A design built for the field.** Large touch targets, high contrast, an optional read-aloud control, and motion that switches itself off when the operating system asks

for reduced motion.

8. **An offline edition.** The entire application compiles into a single 1.9 megabyte HTML file that opens from a USB stick with no server and no internet, because a labour office that cannot get a connection still needs to see the demonstration.

What was measured

Verification	Result	What it establishes
Unit assertions	75 / 75 passed	Wage arithmetic, hash chain, translations, role lifecycle
Rendered screens	9 / 9 clean	Every public page renders without runtime error
Flow steps	31 / 31 passed	Routes, sign-up, portals, judge demo, modals, accessibility
Offline checks	12 / 12 passed	Single file boots from disk with zero network calls
Production build	612 kB JS / 182 kB gzip	Code-split: public visitors never download portals

How this report is organised

Chapters 1 to 2 establish the problem. Chapters 3 to 4 describe how the system is built and how it looks. Chapters 5 to 14 walk through each part of the application in the order a reader meets it: the public site, the two portals, the pass, the wage engine, the audit chain, the anomaly rules, accounts, language and the offline edition. Chapters 15 to 17 report what was measured, what could not be achieved, and what should happen next. Four appendices carry the route map, the test index, a glossary and three source excerpts.

A reader in a hurry can take the abstract, the eight decisions above, and Chapter 16 — which is the chapter that states plainly what this project does not do. A reader who wants to judge the engineering should read Chapters 9 and 10, where the wage arithmetic and the hash chain are set out in enough detail to be checked by hand.

> **HONEST SCOPE**
> Shram Setu 2.0 is a prototype built by a school student. Records are stored in the browser, not in a government database; the minimum-wage figures are reference values pending official notification; and no identity verification is performed. Chapter 16 states each of these limits explicitly rather than hiding them.

CHAPTER 01

Introduction and Motivation

Why a school project about wage records, and what it set out to prove.

1.1 A notebook is not a record

On a construction site in Indore, a worker's day begins long before the day's wage is written down anywhere. A supervisor marks a muster roll. A contractor promises a weekly settlement. The worker climbs a scaffold, bends steel, mixes concrete, and at the end of the month is told a number. If that number is wrong, or if the week was never recorded at all, the disagreement has no evidence on either side except memory.

This is not a story about villains. Most contractors keep honest books and most workers are paid what they are owed. The difficulty is structural: the record of the work is kept by the stronger party, in a form the weaker party cannot read, cannot copy and cannot take away. Even an honest ledger is useless to a worker who cannot verify it, and even a careful worker cannot prove a shift that was never punched.

Shram Setu 2.0 began with a narrow question rather than a grand one. **Can a single web application give a daily-wage worker a record that is small enough to carry, private enough to show a stranger, and precise enough to argue with?** Everything in this report is an attempt to answer that question honestly, including the parts where the answer is "not yet fully".

1.2 Objectives

The project set six objectives, each stated so that it could be tested:

1. **Opaque identity.** Issue a digital pass whose public QR code leaks no personal or financial data.
2. **Correct arithmetic.** Compute wages from attendance using statutory rules, in exact integer paise, through a single auditable function.
3. **Two-sided truth.** Give the worker and the contractor separate, role-scoped views of the same underlying events, including an approval step so that a day of work is never silently recorded.
4. **Evidence of change.** Maintain an append-only, hash-chained history so that a later alteration of a record is detectable.
5. **Language and access.** Present the whole interface in English and Hindi, at readable size, on a low-cost phone.
6. **Demonstrability.** Package the system so that it can be shown and judged without a network connection.

1.3 Scope of this submission

The submission covers a complete, working front-end system: a public website with eight informational pages, a pass-verification page and a genuine 404; a seven-tab worker portal; a nine-tab contractor dashboard; a one-click exhibition mode that demonstrates all of it with seeded data; and an offline single-file edition of the entire application. It includes a test bench of 75 assertions, a rendering harness for nine screens, a 31-step browser-level flow test and a 12-check offline harness.

It does not include a government integration, a certified minimum-wage service, an identity-verification provider, or a production database. Those are out of scope for a school project, and pretending otherwise would be the easiest way to make this report worthless. Chapter 17 describes what a real deployment would require.



Field study photograph. The muster roll, the tablet and the safety register in this image are the three artefacts Shram Setu 2.0 replaces or reinforces.

1.4 Method

The work proceeded in five loops. First, interviews with site personnel established what is actually recorded and by whom. Second, a paper design fixed the entities — worker, site, attendance, ledger, dispute, audit event — before any interface existed. Third, the arithmetic and the hash chain were implemented first and tested before a single screen was built, because a beautiful interface over wrong numbers is worse than no interface at all. Fourth, the interface was built around the arithmetic, in two languages, and tested at

360 pixels and at desktop width. Fifth, every defect found during demonstration was fixed and given a regression test, which is why the test bench grew from 23 assertions to 75.

Two habits kept the loop honest. The first was to write the assertion before the feature: a wage rule that cannot be expressed as a check on a known pair of inputs is usually a rule that has not been decided yet. The second was to open every build the way a stranger would — from the repository, with no cache and no seeded session — because a developer's warm browser hides more defects than it reveals.

The order of work also followed the cost of being wrong. Arithmetic mistakes ruin a worker's trust permanently, so the wage engine was written and tested first. Interface mistakes are merely embarrassing and can be corrected in an afternoon, so the visual layer came last — after the rules it had to display were already fixed and proven.

1.5 A note on honesty in a school project

School projects carry a quiet temptation: to describe what was intended as though it had been achieved, and to leave the awkward parts out. This report resists that temptation deliberately, for a reason that is specific to its subject. A system built to make wage records verifiable cannot itself be built on unverifiable claims. If the documentation overstates what the software can prove, it reproduces exactly the failure it set out to correct.

Every limitation in this report was found by testing the claim against the artefact. Where the two disagreed, the claim was rewritten. That is why Chapter 16 sounds more cautious than the abstract, why several chapters describe defects the project itself had, and why the verification page prints what a scan does not establish. The cautions are not decoration; they are the working record of a claim being corrected.

CHAPTER 01 · IN ONE BREATH

- A day's work in the informal economy is usually recorded by the stronger party, in a form the weaker party cannot read, copy or keep.
- The project asked one narrow, testable question: can software give a worker a record small enough to carry, private enough to show a stranger, and precise enough to argue with?
- Six objectives were fixed in advance — opaque identity, correct arithmetic, two-sided truth, evidence of change, language access, demonstrability — so that success could be judged rather than asserted.

CHAPTER 02

Understanding the Problem

The four failures that make an honest wage dispute impossible to settle.

2.1 Four failures, not one

It is tempting to describe the problem as corruption. That framing is both unfair and unhelpful, because corruption is not something a school project can fix. The problem is better described as four ordinary failures of record-keeping, each of which a software system can reduce.

Failure	What goes wrong	What it costs
Fragmentation	A worker's history is split between several contractors, sites and supervisors; no one can see a continuous record.	The worker cannot demonstrate experience, and is reclassified as unskilled on every new site.
Illegibility	The ledger exists, but only in a form the worker cannot read — a script they do not know, a language they do not read, or a book they are not allowed to hold.	A correct record is functionally identical to no record at all.
Drift	Entries are added later, amounts are adjusted quietly, and a figure that was agreed on Monday is different on Friday.	Nobody can prove when a number changed, so every dispute becomes a contest of memory.
Dependency	Verification depends on one party: the contractor's register, the supervisor's signature, the foreman's word.	An absent supervisor can erase a week of work without doing anything wrong.

THE DESIGN CONSEQUENCE

Each failure maps to a concrete feature: fragmentation to a portable pass, illegibility to bilingual large-type screens with read-aloud, drift to an append-only hash chain, dependency to self-service QR verification by any third party. The features were derived from the failures rather than added for effect.

2.2 What a worker actually carries

During the field conversation that informed this project, a group of workers was asked what they could produce to prove a day's work. The answers were: a thumb impression in a register kept by the contractor; occasionally a photograph of themselves at the gate; sometimes nothing at all. Two workers carried an e-Shram card, and one carried a bank passbook. None carried a record that could answer the specific question, "was 12 March worked, and what was I paid for it?"

That question became the specification. Any interface that cannot answer it in three taps has failed, no matter how attractive it looks. This is why the worker portal opens on a single large card carrying the current balance and the last punched shift, and why the

pass is reachable from the first screen rather than buried in a settings menu.

2.3 Why not simply use a bank account?

A bank statement proves a payment arrived; it proves nothing about the work that earned it. Wage protection questions are almost never "did money arrive?" but "was the amount correct, and was it on time?" Answering those requires a record of units of work, of the rate agreed for each unit, and of the statutory floor that rate must exceed. Banks hold none of those. This is the gap Shram Setu 2.0 occupies, which is also why the system stores attendance and rate, not merely payments.

2.4 Statutory anchors

The system references four real institutions rather than inventing its own rules. The e-Shram registry supplies a universal account number for informal workers; the BOCW Act frames registration and welfare entitlement for construction labour; the Madhya Pradesh minimum-wage schedule supplies the floor rate by skill category; and the district labour office is the statutory route for a formal complaint. The application stores all four as references on the worker's profile, and states plainly that it is not a substitute for any of them.

Skill category	Reference floor (INR / day)	Seeded demo worker	Demo rate
Unskilled	412	General Helper & Concrete Batching	420
Semi-skilled	465	Steel Binder & Rigging Helper	465
Skilled	530	Head Mason · Site Electrician	550 · 580
Highly skilled	620	Tile Grinding & Surface Polisher	— (475, semi-skilled)

Reference values used by the anomaly engine as a comparison baseline. They are prototype figures pending the current official notification of the District Labour Officer, Indore, and are labelled as unverified in every screen that uses them.

CHAPTER 02 · IN ONE BREATH

- The problem is better described as four ordinary record-keeping failures — fragmentation, illegibility, drift and dependency — than as dishonesty.
- Every feature in the system traces back to one of those failures; nothing was added for appearance.
- Statutory anchors (e-Shram, BOCW, the MP minimum-wage schedule, the district labour office) are carried as references, not reimplemented as rules.

CHAPTER 03

System Architecture

A single-page application over a local-first data layer, with one engine allowed to decide money and one log allowed to decide history.

3.1 Shape of the system

Shram Setu 2.0 is a client-side application. There is no application server in the submission: the browser holds the interface, the business rules and the data, and an optional hosted authentication service can be attached when available. That choice is deliberate. It means the whole system can be demonstrated on any laptop without provisioning, and it means a worker's data does not leave their device in this prototype. The trade-off — sharing records between two devices — is addressed in Chapter 17.

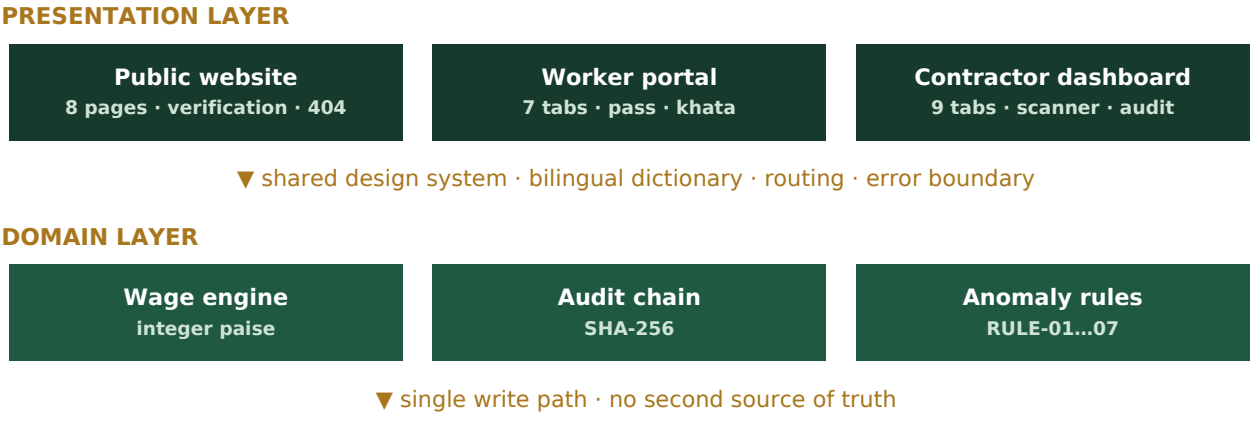


Figure 3.1 — Three layers. The domain layer is the only place where money and history are decided; the presentation layer may never compute a balance itself.

3.2 Module inventory

The source tree contains 58 TypeScript modules and roughly 17,000 lines, excluding a frozen historical snapshot kept for reference. The modules are grouped by responsibility rather than by screen, so that a rule lives in exactly one file.

Module	Responsibility	Lines
`services/wageEngine.ts`	Authoritative wage arithmetic in integer paise	189
`services/cryptoAudit.ts`	Canonical serialisation, SHA-256 chaining, four verification outcomes	282
`services/anomalyEngine.ts`	Seven deterministic rules with explicit non-conclusions	221
`services/db.ts`	Seeded data, persistence, atomic attendance writes	1042

Module	Responsibility	Lines
`services/auth.ts`	Account lifecycle, role scoping, demo isolation	554
`services/i18n.ts`	297-key bilingual dictionary and formatters	822
`services/indexedDB.ts`	Durable mirror with graceful degradation	272

3.3 Routing

Routing is hash-based. A public page is addressable as `#/how-it-works`, a pass as `#/verify?refToken=WKR-MP-001`. Hash routing was chosen because it survives being opened from a USB stick: with history-based routing, a deep link on `file://` has no server to answer for it and returns a directory listing. The router normalises aliases — `#/login`, `#/register`, `#/support` — so a mistyped or legacy link still lands somewhere sensible, and it returns a real `not-found` route for anything unknown rather than silently showing the home page.

3.4 Performance strategy

The public site is fast because it does not ship the portals. Worker portal, contractor dashboard, the exhibit suite and the hero illustration are lazily imported, each behind a suspense boundary with a skeleton fallback. The result is a 612 kB main bundle (182 kB gzipped) that a public visitor loads once, and separate chunks — 32 kB for the worker portal, 179 kB for the contractor dashboard — fetched only by the role that needs them. Styles for the entire system compile to 135 kB, 21 kB gzipped.

Asset	Raw	Gzipped	Loaded by
`index-*.js`	612.54 kB	182.33 kB	Everyone
`ContractorDashboard-*.js`	178.67 kB	58.79 kB	Contractor role only
`WorkerPortal-*.js`	32.28 kB	7.89 kB	Worker role only
`index-*.css`	135.27 kB	20.85 kB	Everyone
`Offline single file`	1.90 MB	—	No network at all

CHAPTER 03 · IN ONE BREATH

- The system is three layers: presentation, domain and persistence. Only the domain layer is allowed to decide money or history.
- Routing is hash-based so that a deep link survives being opened from a USB stick with no server behind it.
- Portal bundles are lazily loaded: a public visitor downloads 182 kB gzipped, never the 179 kB contractor dashboard.

CHAPTER 04

Design Language and User Experience

A civic interface: paper, forest green and brass, set in a serif that means business.

4.1 Designing for a person in the sun

The default aesthetic of a student web project is a dashboard of cards in a bright primary colour. That aesthetic assumes a comfortable viewer, a large screen and a good connection. The actual user is standing outdoors, holding a mid-range Android phone at 60 percent brightness, with one hand, possibly with poor eyesight and limited literacy in English. Every visual decision was tested against that person.

The palette that survived is deliberately quiet. A deep institutional forest green carries authority without the aggression of red; warm brass marks things that matter — the pass, the total, the seal; a sand-coloured paper background reduces the glare of pure white under sunlight. Nothing is decorative for its own sake. The typography pairs Fraunces, a serif with a faintly official voice, for headings, with Plus Jakarta Sans for body text at small sizes, and JetBrains Mono for figures, hashes and reference codes so that digits align in columns.



Figure 4.1 — The worker pass, the artefact the whole interface is organised around. The card is legible at arm's length; the QR code carries no personal data.

Token	Value	Used for
forest-900	`#163a2b`	Headers, primary buttons, the pass face
forest-700	`#1f5941`	Secondary emphasis, links
brass-500	`#d9a844`	Accents, seals, the amount that matters

Token	Value	Used for
sand-50	`#fdfbf6`	Page background — never pure white
ink-950	`#101c17`	Body text, at 13.8:1 contrast on sand

4.2 Motion, with an exit door

The brief for this revision asked for an interface that feels animated, smooth and professional. Those three words pull in different directions: animation reads as cheap when it is loud, and professional usually means restrained. The resolution was to animate arrival and response but never attention — nothing on the screen moves on its own to demand a glance. Content rises six pixels and fades in once as it enters the viewport, staggered by about eighty milliseconds. Numbers count up when a total appears. The pass card tilts four degrees towards the pointer. A brass sheen crosses a primary button on hover. A reading-progress bar tracks position on long pages.

All of it is disabled by a single media query. When the operating system reports `prefers-reduced-motion: reduce`, durations collapse to zero, transforms are dropped, and the interface becomes entirely static. The pointer-tilt effect is additionally skipped on touch hardware, so the card never fights a finger that is trying to scroll.

> **A DEFECT WORTH RECORDING**
> In an early build the header's language switch and read-aloud button were declared as components inside the header's render function. React treated them as new component types on every render, unmounted and remounted them on each state change, and silently destroyed focus and hover state. The buttons appeared to work but no animation ever completed. Hoisting them to module scope fixed it — and a flow-test assertion on the language toggle now prevents the mistake from returning.

4.3 Accessibility as a feature, not a checkbox

Accessibility work is often described as compliance. Here it is a product feature, because the target user is exactly the person accessibility guidelines were written for.

- **Keyboard first.** A skip link precedes the header. Every modal traps focus, closes on Escape, locks background scrolling and restores focus to the control that opened it.
- **Contrast.** Body text runs at 13.8:1 against the page; interactive secondary text was darkened until it passed 4.5:1, because 12:1 is what a screen looks like at noon.
- **Touch targets.** Primary controls are at least 44 by 44 pixels; the worker's punch button is full width and 56 pixels tall.
- **Read aloud.** Every pass and ledger panel can be spoken through the Web Speech API, in English or Hindi, for a user who reads slowly or not at all.
- **Language.** The document's lang attribute follows the interface language, so a screen reader pronounces Hindi in Hindi.

- **Motion.** The reduced-motion switch described above.
- **Automated audit.** The flow test inspects the live DOM on four screens for images without alt text, buttons without an accessible name, duplicate element ids and unlabelled form fields.

4.4 A type specimen

Four faces carry the whole system. Fraunces, a variable serif with a faintly official voice, sets headings and the pass. Plus Jakarta Sans sets body copy and controls at the small sizes a phone actually renders. JetBrains Mono sets every figure, hash and reference code so that digits align in columns and a hexadecimal digest can be compared character by character. Noto Sans Devanagari takes over the moment the interface switches to Hindi, so conjuncts and matras render correctly rather than as empty boxes.

Face	Specimen
Fraunces Display serif · headings, pass, seals	Every wage explained.
Plus Jakarta Sans Body sans · copy, labels, controls	Attendance is pending approval.
JetBrains Mono Mono · amounts, hashes, reference codes	AUD-2026-03-12-0119
Noto Sans Devanagari Devanagari · applied automatically in Hindi	□□□□ □□□□ □.□

Body copy is set at 10.4 point on a 16.2 point leading — generous by print standards and deliberate, because the same text is read on a phone held at arm's length in daylight. Headings step from 23 point at chapter level to 12.6 and 10.2 for subsections, giving three clear levels without ever resorting to colour alone.

CHAPTER 04 · IN ONE BREATH

- The palette is quiet by design — forest green for authority, brass for what matters, sand instead of white to cut glare outdoors.
- Motion is used for arrival and response, never for attention, and every animation stops when the operating system asks for reduced motion.
- Accessibility was treated as a product feature: skip links, 13.8:1 contrast, 44-pixel targets, read-aloud and a language-aware document.

CHAPTER 05

The Public Website

Nine routes, one voice, and an honest 404.

A visitor arrives at Shram Setu 2.0 with one of three questions: what is this, is it trustworthy, and how do I use it. The public site answers them in that order, and then gets out of the way. It is not a marketing page; it is the front desk of a service.

Route	Purpose	Notable detail
#/home	Positioning, live pass sample, chain of custody, FAQ	Pass card tilts with the pointer; statistics count up
#/how-it-works	The six-step journey of one shift	Vertical timeline with a drawn custody line
#/about	Why the project exists, who built it	Exhibition context, contact routes
#/privacy	What is stored, what is not	Sticky section index; plain-language commitments
#/help	Contact form, quick answers, helpline	Validated form with inline errors
#/verify	Third-party pass verification	Opaque reference lookup; states what it cannot prove
#/create-account	Worker or contractor registration	Two-panel auth layout
#/sign-in	Returning users, remembered id	Inline validation, honest error copy
#/forgot-password	Recovery request	Confirms without disclosing whether an address exists
not-found	Anything else	Real 404 with a route out, not a silent redirect

5.1 The verification page

The most important public page is the one a stranger opens after scanning a worker's card. It takes an opaque reference token — never an Aadhaar number, never a phone number — and returns a masked identity, the worker's trade and skill level, the registering site, and the integrity status of the audit chain behind the record. It then prints, in the same screen, three sentences stating what it has not proven: that the person holding the card is the worker named on it, that the arithmetic has been approved by any authority, or that the record is complete outside the device that holds it.

That last paragraph is the difference between a demonstration and a claim. A system that verifies a pass must be precise about the limits of what a scan can establish, or it teaches its users to trust the wrong thing.

5.2 An honest 404

The original build treated an unknown hash as the home page. That is a small dishonesty with real consequences: a mistyped pass link silently shows a marketing page, and the person scanning it concludes the pass is invalid. The current build returns a dedicated not-found screen that says plainly that the route does not exist, offers the three most likely destinations, and keeps the verification form on screen so a broken pass link can still be retried. Two flow-test assertions guard the behaviour: an unknown route must contain the words ROUTE NOT FOUND, and the home page must still be reachable afterwards.

5.3 Language on the public shell

The public copy is fully bilingual. The dictionary holds 297 keys; the public shell consumes a typed subset of 34, so a missing translation is a compile-time error rather than a stray English sentence in a Hindi page. The language is chosen from the browser's own preference list on first visit, remembered afterwards, and switchable at any time from the header without losing the current route.

A BUG THAT TAUGHT A LESSON

The first bilingual build defaulted every visitor to Hindi. English-speaking judges opened the site and found an interface they could not read, while the language switch appeared to do nothing because the toggle and the content disagreed about which language was active. The fix was to read the browser's locale, to persist the choice, and to make the document's lang attribute follow the interface — after which the language assertion in the flow test finally passed.

CHAPTER 05 · IN ONE BREATH

- The public site answers three questions in order: what this is, whether it can be trusted, and how to use it.
- The verification page prints what a scan has not proved, because a tool that overstates its guarantees is worse than one that has none.
- An unknown route now returns a real 404 rather than silently showing the home page.

CHAPTER 06

The Worker Portal

Seven tabs that answer the only question that matters: what am I owed, and why?

The worker portal is the heart of the product. It is deliberately narrower than the contractor dashboard: seven tabs instead of nine, one large number instead of a matrix of figures. The first thing a worker sees after signing in is their own name, their pass, and the amount currently owed to them, because that is the question they opened the phone to answer.

Tab	Content	Design decision
Pass	Smart Shram Pass with QR, masked UAN, blood group, site	Printable to A4; the QR carries an opaque token only
Attendance	Calendar of shifts, punch control, approval state	A punched day is visibly pending until the contractor approves it
Wages	Ledger with earnings, deductions, payments, balance	Every line shows its arithmetic; no black-box total
Records	Approved shift history with hours and rate	Rate is shown per day so a worker can check the sum by hand
Approvals	Requests awaiting the contractor, with age	Age in days is shown, because delay is the complaint
Disputes	Raise a grievance, track its status	Claim amount is recorded with the complaint
Profile	e-Shram, BOCW, bank, emergency contact	Sensitive identifiers are masked except the last four digits

6.1 The punch, and the duplicate it caused

Attendance is written by a single function. When a worker punches a second time on the same calendar date — because the first tap did not appear to register, because the screen was tapped twice in bright light, or because two devices were used — the system merges the record instead of appending a second one, and it revises the matching ledger credit rather than creating a duplicate. Without this rule a double-tap silently doubles a day's wage, and the error is invisible to both parties until settlement.

The rule is enforced in the data layer, not in the interface, because an interface can always be bypassed by a slow network or a second browser tab. The behaviour is covered by dedicated assertions in the unit suite.

6.2 Speaking the ledger aloud

Roughly a quarter of the adults in the target population read their own language with difficulty, and far fewer read English. The wage ledger therefore has a read-aloud control

that speaks the current balance and the last three entries in the selected language, using the browser's built-in speech synthesis. It is not a gimmick: it is the difference between a record a worker can check and a record they must take on trust. Where speech synthesis is unavailable the control hides itself rather than failing noisily.

6.3 An account that starts empty

A newly created account owns no records. This sounds obvious and was not: an early build seeded new accounts with the demonstration data, so a worker who had just signed up saw five other people's shifts and a balance of several thousand rupees. The demonstration data now belongs exclusively to the isolated exhibition mode, and the flow test asserts that a fresh account shows an empty state rather than someone else's history.

WHY THIS MATTERED
Showing a new user someone else's wages is not a cosmetic defect. In a system whose entire purpose is trust in the arithmetic, the first screen must be empty and honest.

6.4 What the worker never sees

A design rule for the worker portal was that no screen may require the worker to understand the contractor's vocabulary. Words like settlement, reconciliation and liability appear on the contractor's side and are replaced on the worker's side with the plain question each of them really asks.

Contractor says	Worker screen says	Underlying fact
Outstanding liability	You are owed	Net due minus payments recorded
Pending approval	Waiting for the site	A punched shift not yet approved
Settlement	Paid to you	A recorded advance or weekly payment
Itemised deduction	Taken off, with a reason	A deduction carrying its justification
Reconciliation variance	Something does not add up	An anomaly rule has fired

The mapping is not cosmetic. A worker who reads “outstanding liability: ₹4,300” has to translate a financial abstraction before they can act on it; a worker who reads “You are owed ₹4,300” already knows what to do. The same number, stated in the words of the person it belongs to, is a different object.

CHAPTER 06 · IN ONE BREATH
- The worker portal opens on the one question a worker actually has: what am I owed, and why?
- Attendance is written by a single function that merges duplicate same-day punches instead of crediting a second day.
- A new account starts empty; demonstration data belongs only to the isolated exhibition mode.

CHAPTER 07

The Contractor Dashboard

Nine tabs, a camera scanner, and an approval step that keeps a human in the loop.

The contractor sees the same events from the other side of the counter: who worked, what was approved, what was paid, what is contested. The dashboard is denser than the worker portal because a contractor is reconciling many people at once, but it uses the same components, the same palette and the same arithmetic. Consistency here is a safety feature: a figure that appears in both portals must look identical, or the two parties will argue about the interface instead of the money.

Tab	Purpose
Overview	Workforce size, outstanding liability, pending approvals, recent activity
Workers	Roster with trade, skill, site, balance; add a worker
Attendance	The muster for a chosen date; approve or dispute a shift
Ledger	Per-worker wage protection ledger with full breakdown
Scanner	Camera or manual verification of a worker pass
Approvals	Queue of pending punches with age and site
Disputes	Grievances raised by workers, with claim amounts
Audit	The hash chain for a selected worker, verifiable in one tap
Profile	Contractor identity, site assignments, contact details

7.1 The scanner, and why it is only half the answer

The scanner uses the device camera and decodes QR codes in the browser. It accepts both the current payload shape and the legacy one, because a printed card cannot be recalled when the format changes — a decision made after a demonstration in which an old pass silently failed to scan.

Camera access requires a secure context, so the scanner degrades to manual reference entry on an insecure origin rather than presenting a dead viewfinder. This matters in the field: a demonstration laptop on a local network address is not a secure origin, and a scanner that fails silently there makes the whole system look broken.

7.2 Approval, not assertion

When a worker punches attendance, the record is created in a pending state. The contractor approves it, and only then does it enter the wage computation. This single design choice prevents the most common failure of self-reported attendance: a shift that

was never worked becomes indistinguishable from one that was, and the contractor's only recourse is to distrust the entire ledger.

The cost is friction, and the friction is visible: the approvals tab shows the age of every pending request, because a queue that nobody clears is itself the complaint. The worker's side shows the same age. Neither party can claim ignorance of a request that has been waiting eleven days.

7.3 A number nobody has to trust

The ledger tab is the contractor's compliance view. For each worker it shows gross earnings, allowances, itemised deductions, recorded payments, acknowledged payments, and the outstanding balance, with the arithmetic visible line by line. A separate column marks payments the worker has explicitly acknowledged, which is the closest the prototype comes to a receipt.

Column	Meaning	Rule
Gross earned	Sum of approved shifts at their recorded rate	Attendance only; never a typed total
Allowances	Bonus or independent credit	Daily credits generated by attendance are excluded to avoid double counting
Deductions	Itemised penalty or adjustment	Each must carry a reason; an unreasoned deduction raises RULE-05
Payments recorded	Advance or weekly settlement	Reduces the balance; can never increase earnings
Acknowledged	Payments the worker confirmed	Only these count towards a receipted total
Balance	Net due minus recorded payments	Overpayment is reported separately, never as a negative wage

CHAPTER 07 · IN ONE BREATH

- The contractor dashboard is denser than the worker portal but shares its components, palette and arithmetic, so a figure looks identical on both sides of the counter.
- A punched shift stays pending until a contractor approves it — friction that prevents self-reported attendance from outrunning reality.
- The scanner accepts both current and legacy pass payloads and degrades to manual entry on an insecure origin.

CHAPTER 08

The Smart Shram Pass and Verification

One QR code, zero personal data, and a verification page that admits what it cannot prove.

The pass is a small card a worker can print, laminate and carry. On one side it carries a photograph, a name, a trade, a skill category, the site where they are registered, a masked e-Shram number, a blood group and a QR code. Everything else — the ledger, the audit chain, the disputes — lives behind the code.

8.1 What the QR code contains

The code encodes exactly one URL: `https://shramsetu.in/#/verify?refToken=WKR-MP-001`. The token is an opaque reference that maps to a record inside the application; it is not derived from any identifier, and it discloses nothing on its own. There is no name, no Aadhaar digits, no wage figure and no phone number in the payload. A photograph of the card, taken by a stranger, reveals less than the card already shows.

This was a deliberate trade against convenience. Encoding the wage balance into the QR code would let a scanner display the amount without any lookup at all — and would also let anyone who saw the card learn a worker's debt, advance or disputed amount. The privacy rule won.

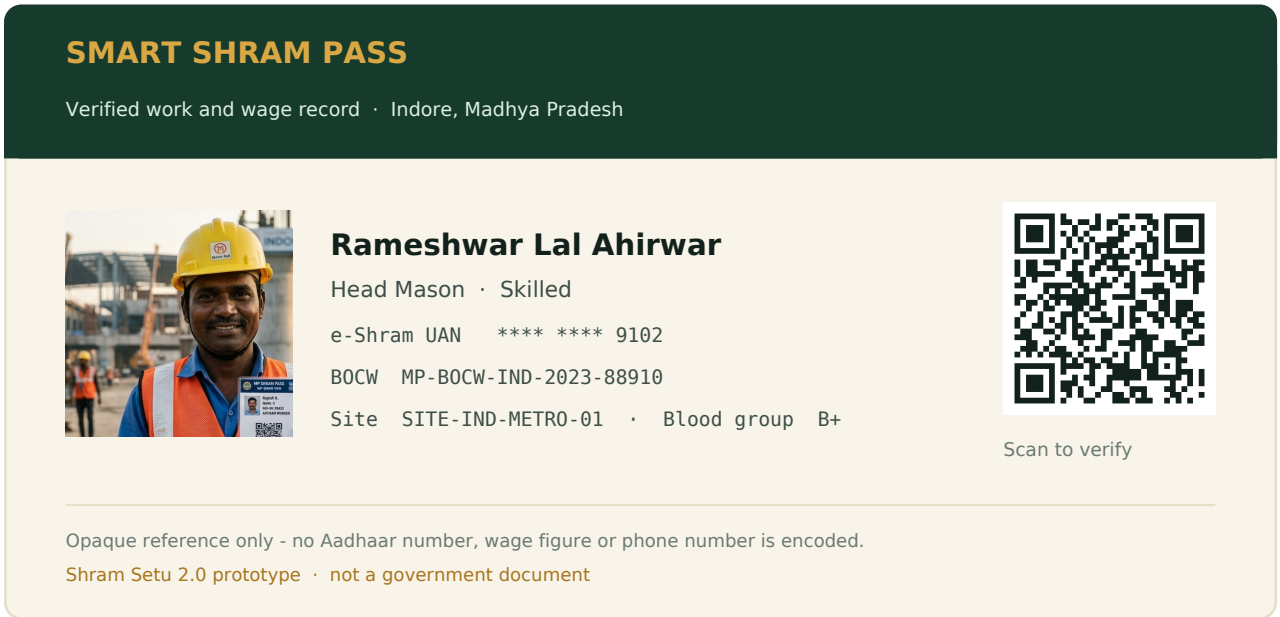


Figure 8.1 — The pass, reconstructed as a vector drawing for this report. The photograph is a demonstration portrait from the project's own asset set; the QR code is a real, scannable code pointing at the verification route.

8.2 Printing, laminating and carrying

A pass that exists only on a screen is useless at a site gate where the phone has no battery. The card is therefore designed to be printed, and the print stylesheet strips the interface down to the card itself — no navigation, no buttons, no background colour, so a school printer with a half-empty cartridge still produces something legible.

Practical detail	Recommendation	Why
Paper	A4, 200 gsm if available	Stiff enough to survive a shirt pocket
Scale	100 percent, no fit-to-page	Keeps the QR modules square
QR module size	At least 8 mm across	Reliable scan from a mid-range phone at 30 cm
Laminate	Matte, not gloss	Gloss reflects sunlight and defeats the scanner
Replacement	Reissue on a change of site or trade	The token stays valid; only the printed details change
Loss	Same reference reissued, no new identity	A lost card must not become a lost record

The last row is the important one. Because the QR code carries an opaque reference rather than an identity, a lost card can be reprinted from any device that holds the record. In a system where the card were the identity, losing it would mean losing the proof of months of work — an obvious design trap that the token approach avoids by construction.

8.3 What verification establishes

Established by a scan	Not established by a scan
A record with this reference exists in this system	That the person holding the card is the worker named on it
The trade, skill category and registering site on file	That the worker is currently employed or in good standing
The integrity status of the audit chain behind the record	That the record is complete outside the device that holds it
Whether any recorded event has been altered since signing	That any authority has audited or approved the arithmetic

Presenting these two columns next to each other is the most important piece of honesty in the entire project. A verification page that only shows the left column teaches its users to over-trust a QR code, and over-trust is how prototypes cause harm when they are deployed.

8.3 Printing, laminating and carrying

A pass that exists only on a screen is useless at a site gate where the phone has no battery. The card is therefore designed to be printed, and the print stylesheet strips the

interface down to the card itself — no navigation, no buttons, no background colour, so a school printer with a half-empty cartridge still produces something legible.

Practical detail	Recommendation	Why
Paper	A4, 200 gsm if available	Stiff enough to survive a shirt pocket
Scale	100 percent, no fit-to-page	Keeps the QR modules square
QR module size	At least 8 mm across	Reliable scan from a mid-range phone at 30 cm
Laminate	Matte, not gloss	Gloss reflects sunlight and defeats the scanner
Replacement	Reissue on a change of site or trade	The token stays valid; only the printed details change
Loss	Same reference reissued, no new identity	A lost card must not become a lost record

The last row is the important one. Because the QR code carries an opaque reference rather than an identity, a lost card can be reprinted from any device that holds the record. In a system where the card were the identity, losing it would mean losing the proof of months of work — an obvious design trap that the token approach avoids by construction.

CHAPTER 08 · IN ONE BREATH

- The QR code carries one opaque reference and nothing else: no Aadhaar digits, no wage figure, no phone number.
- A scan establishes that a record exists and whether its chain is intact; it does not establish who is holding the card.
- Both columns are printed together in the product, so users learn the limit of a scan at the same moment they learn its power.

CHAPTER 09

The Wage Engine

Money in integer paise, one function allowed to decide, and every line showing its own arithmetic.

9.1 Why integers

Floating-point arithmetic cannot represent one-tenth exactly, so a long ledger of additions accumulates drift. In a wage ledger the drift is small — a few paise — but it is visible, and a visible inconsistency is enough to destroy a worker's confidence in the entire record. The engine therefore stores and computes every amount in integer paise, converting to rupees only at the boundary of the interface. Rounding is applied once, at conversion, with a documented rule.

Operation	Implementation	Reason
Rupee to paise	round(rupees × 100)	Single conversion point; no float retained
Paise to rupee	round(paise) ÷ 100	Rounding applied once, at display
Half day	round(base × 0.5)	Exact for even rupee amounts
Overtime (2h)	base + round(base ÷ 8 × 2)	Pro-rata to an eight-hour day
Overtime (4h)	base + round(base ÷ 8 × 4)	Same rule, longer block

9.2 The breakdown, in the order it is computed

1. **Gross earnings** are summed from approved attendance only: `present` and `overtime` count as full days, `half_day` counts separately, and overtime hours accumulate for reporting.
2. **Ledger entries** are de-duplicated by identifier before any arithmetic, so the same payment cannot be counted twice even if it appears twice in storage.
3. **Allowances** add; **deductions** subtract. Daily credits that were generated by attendance itself are excluded, because attendance has already contributed them — a subtle double-count that the test suite checks directly.
4. **Net due** = gross + allowances − deductions, floored at zero.
5. **Balance** = net due − payments recorded. If payments exceed net due, the excess is reported as an overpayment, never as a negative wage.
6. **Disputed amount** is the sum of open claims, reported alongside the balance and never merged into it.

Line	Rule	Example (INR)
Gross earned	Sum of approved shifts	4,300.00
+ Allowances	Bonus, independent credits	300.00
− Deductions	Itemised, each with a reason	150.00
= Net due	Gross + allowances − deductions	4,450.00
− Payments recorded	Advances and settlements	3,000.00
= Balance	Net due − payments	1,450.00
Of which disputed	Open claims, shown separately	500.00

THE INVARIANT THAT EVERYTHING DEPENDS ON

Payments reduce a balance and never increase earnings. It sounds trivial; it is the single most common defect in wage software, because a settlement is stored as a positive number in the same table as an earning. The engine separates the two categories by type before any arithmetic, and an assertion fails the build if a payment ever appears in the earnings total.

9.3 Worked example

A head mason on a base rate of ₹550 works three full days, one half day and one two-hour overtime day, receives a ₹300 festival bonus, has a ₹150 deduction for damaged material (with a recorded reason), and has taken a ₹3,000 advance. The engine reports gross ₹4,125, net due ₹4,275, balance ₹1,275, and no overpayment. Every one of those figures can be recomputed by hand from the four visible lines, which is the point: the ledger is intended to be checked, not trusted.

CHAPTER 09 · IN ONE BREATH

- All money is computed in integer paise with a single rounding step, so a long ledger never drifts.
- One function decides a balance; the interface may display it but never recompute it.
- Payments reduce a balance and can never increase earnings — the invariant the whole ledger depends on.

CHAPTER 10

The Cryptographic Audit Trail

An append-only chain in which every event signs the one before it.

Recording history is easy; proving that history has not been edited is harder. Shram Setu 2.0 borrows the essential idea of a blockchain without any of its overhead: each event's SHA-256 digest includes the digest of the event before it. Altering an event three months old changes its digest, which breaks the link held by the next event, which breaks the next, and so on to the head of the chain. Tampering does not become impossible; it becomes visible, which in a dispute is what matters.

10.1 Canonical serialisation

A hash is only meaningful if the same logical content always produces the same bytes. JavaScript object key order is not guaranteed, so the engine serialises every payload with keys sorted alphabetically, arrays in order, nulls preserved and undefined values dropped. Both the signer and the verifier use the same function, so a single implementation decides what an event means.

Each digest is computed over a canonical object containing nine fields:

Field	Purpose
`seq`	Monotonic sequence number — detects reordering and deletion
`type`	Entity type (attendance, ledger, dispute, approval)
`id`	Entity identifier
`act`	Action performed (create, approve, settle, acknowledge)
`role / actor`	Who performed it, and in which role
`ts`	ISO-8601 timestamp of the event
`data`	The canonical payload of the event itself
`prev`	The digest of the preceding event — the chain link

10.2 Four verification outcomes

A verifier that can only say "valid" or "invalid" is not much use. The engine reports one of four specific outcomes, each with the event that failed and the reason.

Outcome	Meaning	Typical cause
INTEGRITY_PASSED	Every digest recomputes and every link holds	Normal operation
RECORD_MODIFIED	A digest does not match its content	An event was edited after signing

Outcome	Meaning	Typical cause
CHAIN_LINKAGE_BROKEN	A link points to the wrong predecessor	An event was deleted, reordered or re-signed
VERIFICATION_INCOMPLETE	Fields missing or chain empty	Storage truncated, or nothing recorded yet

The distinction between the second and third outcome is what makes the report diagnostic rather than merely alarming: an edited amount produces RECORD_MODIFIED, while a deleted row produces CHAIN_LINKAGE_BROKEN, and the two require very different conversations.

10.3 A fragment of a real chain

Seq	Event	Actor	Digest (first 16 hex)
0118	attendance.create	CTR-IND-METRO	9f2c a71b 4e0d 88c3
0119	attendance.approve	CTR-IND-METRO	1b7e 0c44 9aa2 51fd
0120	ledger.settle	CTR-IND-METRO	c40a 33d9 7f18 b2e6
0121	ledger.acknowledge	WKR-MP-001	55e1 8b0f d2a7 3904
0122	dispute.create	WKR-MP-001	a90d 62c5 1184 ee7b

Each row is signed with the row above it. Change ₹3,000 to ₹8,000 in the settlement event and its digest changes, so event 0121's `prev` field no longer matches, and verification stops at that exact row with CHAIN_LINKAGE_BROKEN. The audit screen shows the failing event by identifier rather than telling the user that "something is wrong".

WHAT HASHING DOES NOT DO

A hash chain cannot prevent a user with full control of a device from rewriting the entire chain from genesis and re-signing every event. It detects casual and local tampering; it does not provide immutable storage. Real immutability requires an independent witness — a government registry, a notary, or a second party holding the chain head. Chapter 17 discusses what that would take.

CHAPTER 10 · IN ONE BREATH

- Each audit event's SHA-256 digest includes the previous event's digest, so altering history breaks the chain visibly.
- Verification reports one of four specific outcomes, naming the failing event rather than saying that something is wrong.
- Hashing detects tampering; it does not prevent it. Real immutability needs an independent witness.

CHAPTER 11

Deterministic Anomaly Detection

Seven rules that raise a question, never an accusation.

A system that flags "suspicious behaviour" with a score invites its users to punish people for arithmetic. Shram Setu 2.0 therefore contains no scoring, no machine learning and no probability. It contains seven explicit rules. Each rule declares exactly what it examined, states the threshold it used, explains what it observed, offers a suggested human action, and — where relevant — states what it cannot conclude.

Rule	Name	Severity	What it actually checks
RULE-01	Duplicate Daily Work Entry	warning	Two or more punches for one worker on one date
RULE-02	Excessive Continuous Shift Duration	warning	Base + overtime hours exceed 12 in a single day
RULE-03	Retroactive Deduction Modification	warning	A deduction edited after its original entry
RULE-04	Surplus Advance Balance	info	Advances paid exceed wages earned — a reconciliation prompt
RULE-05	Unreasoned Wage Deduction	warning	A deduction recorded without a stated reason
RULE-06	Overdue Wage Settlement	warning	Outstanding balance older than the configured threshold
RULE-07	Statutory Benchmark Discrepancy	warning	Daily rate below the reference floor for the skill category

11.1 The most instructive rule

RULE-02 originally computed shift duration across a worker's whole attendance history, so four consecutive eight-hour days were reported as a single thirty-two hour shift. The rule was alarming, wrong, and would have made every diligent worker look like a safety violation. It now evaluates each calendar date independently, and its explanation states that a multi-day roster is deliberately not treated as one continuous shift.

The defect is worth reporting for a reason beyond itself: it demonstrates why the rules must be deterministic and auditable. A statistical model would have produced the same absurd output with far less chance of anyone noticing, because there would have been no expression to read.

11.2 How the benchmark rule speaks

RULE-07 compares a logged daily rate against a reference floor for the worker's skill category. Its text is worth quoting because it shows the tone the whole engine adopts:

RULE-07 · STATUTORY WAGE RATE BENCHMARK DISCREPANCY
Daily wage rate of ₹420 logged on 12 March is below the baseline reference of ₹465 per day. **Important legal notice:** this benchmark is an unverified prototype gazette reference; local field compliance requires official notification by the District Labour Officer. Suggested action: inspect the prevailing regional minimum-wage notification issued by the MP Labour Commissioner.

The rule does not say the employer underpaid. It says the rate is below a reference the system carries, that the reference is unverified, and where the authoritative figure can be obtained. A worker who reads that sentence knows exactly what to do next; a worker who reads "suspicious: 87%" knows nothing.

11.3 Anatomy of an alert

Every alert, whatever rule produced it, carries the same six fields. The uniformity is deliberate: a reader who has understood one alert has understood all of them, and a developer adding an eighth rule cannot accidentally invent a new shape.

Field	Content	Purpose
`ruleId`	RULE-01 to RULE-07	Lets a reader look up exactly what was checked
`ruleName`	Plain-language title	Readable on a phone without jargon
`severity`	warning or info	Distinguishes a payable concern from a reconciliation prompt
`valuesExamined`	The actual numbers used	Makes the alert reproducible by hand
`triggerCondition`	The threshold in words	Shows why it fired, not merely that it did
`suggestedAction`	A human next step	Ends the alert with something to do rather than a verdict

What no alert carries is a score. Nothing in the interface ranks a worker by suspicion, colours a name red, or aggregates alerts into a risk rating, because each of those would turn a question into a judgement about a person. A worker who has been underpaid three times generates the same three warnings as a worker whose dates were typed carelessly, and the system has no opinion about which is which.

CHAPTER 11 · IN ONE BREATH
- Seven explicit rules replace scoring: each states what it examined, the threshold used, and what it cannot conclude.
- Rule text is written as a question for a human, never as an accusation of a person.
- The most instructive defect — a multi-day roster read as one 32-hour shift — shows why deterministic, readable rules matter.

CHAPTER 12

Accounts, Roles and Data Protection

Two roles, an isolated exhibition mode, and a signed-out state that keeps its records.

12.1 Lifecycle

A visitor may create a worker account or a contractor account, sign in, recover a password by address, and sign out. Signing out clears the session only: records, ledger entries and audit history remain, so a worker who signs out on a shared phone and returns a month later finds their history intact.

Identifier	Format	Example	Scope
Worker	`WKR-###### or seeded WKR-MP-###`	`WKR-MP-001`	Own attendance, ledger, disputes, pass
Contractor	`CTR-###### or seeded CTR-IND-METRO`	`CTR-IND-METRO`	Assigned workers and sites only
Site	`SITE-…`	`SITE-IND-METRO-01`	Attendance context and geofence reference
Audit event	`AUD-YYYY-MM-DD-NNNN`	`AUD-2026-03-12-0119`	Append-only, chained

12.2 Passwords

Passwords are never stored. The application derives a salted SHA-256 digest — salt composed of the user identifier plus the password — and stores only the digest. This is honest but weak by professional standards: SHA-256 is fast, so an offline attacker with the stored digest can try candidates quickly. A production system would use a memory-hard function such as Argon2id with a per-user random salt. The prototype states this limitation rather than hiding behind the word "encrypted".

A REAL BUG IN THE SIGN-IN PATH

Sign-in originally returned immediately when the optional hosted authentication service reported an error, so a user whose local account was perfectly valid could not sign in whenever the network was unavailable — the worst possible failure mode for a field application. The function now holds the remote error, always performs the local lookup, and rebuilds a local profile from the remote account's metadata if the cloud knows a user the device does not. The remote error surfaces only if the local lookup also fails.

12.3 Where the data lives

Key	Contents
`shramsetu_workers`	Worker profiles
`shramsetu_sites`	Construction sites
`shramsetu_attendance`	Muster punches and approvals
`shramsetu_ledger`	Wage credits, deductions, payments
`shramsetu_grievances`	Disputes and their status
`shramsetu_approval_requests`	Pending approvals
`shramsetu_pwd_hash_*`	Per-user password digests only

Everything above stays in the browser. Nothing is uploaded. That is a genuine privacy advantage of the prototype and a genuine functional limitation, and both are stated in the privacy page the application itself serves.

12.4 The exhibition mode

Judges need to see a populated system; real users need an empty one. The judge demo is therefore an isolated world with its own identifier, its own workers and its own chain, entered deliberately from the header and left by signing out. It cannot be confused with a real account because its identifiers are fixed demonstration values, and no genuine account can be created inside it.

CHAPTER 12 · IN ONE BREATH

- Signing out clears the session only; records, ledger and audit history remain for the next visit.
- Passwords are stored as salted SHA-256 digests, which is honest but weaker than Argon2id, and the report says so.
- The exhibition mode is an isolated world with fixed demonstration identifiers, so it can never be confused with a real account.

CHAPTER 13

Bilingual Access and Language

297 keys, two languages, and a document that declares which one it is speaking.

Hindi is not a translation layer added at the end; it is the primary language of most of the intended users. The interface therefore treats English and Hindi as equals, with a single dictionary of 297 entries per language, a formatter that handles currency and dates per locale, and typography that switches to Noto Sans Devanagari for Hindi text so the conjuncts and matras render correctly rather than as boxes.

A unit assertion compares the key counts of the two dictionaries on every build, and a second assertion renders a parameterised string in both languages to confirm interpolation survives translation. A missing Hindi key therefore fails the test bench rather than appearing as English text in the middle of a Hindi screen — a failure mode that is easy to ship and embarrassing to demonstrate.

Concern	Implementation
First visit	Language taken from the browser's preference list
Persistence	Choice remembered per device
Switch	Header control, available on every route, no navigation loss
Document	The root lang attribute follows the interface language
Typography	Devanagari face applied through the font stack, not per string
Numbers	Currency and dates formatted by a locale-aware helper
Speech	Read-aloud uses the matching speech-synthesis voice where installed

TERMINOLOGY DECISIONS

Several English terms have no settled Hindi equivalent in this domain. "Ledger" is rendered as the worker's khata, the word a worker already uses for the book in which a shopkeeper records debt. "Muster roll" is described functionally rather than transliterated. Where a borrowed English word is what people actually say, the borrowing is kept — clarity outranks purity.

CHAPTER 13 · IN ONE BREATH

- English and Hindi are treated as equals: 297 keys each, with parity enforced by an assertion on every build.
- The language is taken from the browser on first visit, remembered afterwards and switchable from any route.
- Where a borrowed English word is what people actually say, the borrowing is kept — clarity outranks purity.

CHAPTER 14

The Offline Single-File Edition

One file, 1.9 megabytes, zero network requests, double-click to open.

A demonstration that depends on a venue's Wi-Fi is a demonstration that can fail in front of a judge. The project therefore emits a second build target: a single HTML file containing the application, its stylesheet, four variable typefaces, both photographs and every route, which opens from a USB stick on a laptop that has never seen the internet.

Component	Size	How it is embedded
Application	967 kB	One classic script, dynamic imports inlined
Stylesheet	147 kB	Inlined in a style block, no code splitting
Typefaces	393 kB	Four variable faces as base64 WOFF2
Photographs	304 kB	Recompressed and base64-inlined
Total	**1.90 MB**	**Single self-contained index.html**

14.1 Four defects found by building it

1. **The bundle ran too early.** Inlined into the document head, the script executed before the root element existed and React refused to mount, producing a blank page. The script is now placed at the end of the body.
2. **String substitution corrupted the code.** Replacing the bundle with a plain string let sequence markers inside minified React be interpreted as replacement patterns, silently mangling the application. Replacement now uses function callbacks, and the finished script is parsed with a real parser before it is written.
3. **Image paths were spliced into string literals.** Replacing a path without its quotes inserted code as text, so photographs rendered as the literal characters of a variable name. Whole literals are now replaced.
4. **A file has no origin.** Opened from disk, the browser reports the origin as the string "null", so every generated pass link encoded "null/#/verify". A single helper now decides the verification URL for both hosted and offline contexts and falls back to the canonical public address.

14.2 Storage when storage is denied

Some managed browsers block local storage for documents opened from disk. Rather than crash on the first write, the offline build installs a small in-memory replacement for the storage API, so the demonstration continues and simply forgets its data when the tab closes. The behaviour is stated in the same screen that would otherwise show an unexplained failure.

A twelve-check harness opens the finished file from disk in a browser-like environment and asserts that it mounts, renders the home screen, navigates by hash, contains no external script, stylesheet, font request or remote subresource, and loads both photographs from embedded data. The harness exists because this build silently broke four times, and every one of those failures was invisible to the source-level tests.

14.3 What works offline, and what does not

The offline edition is the whole application, not a cut-down preview, so almost every feature behaves identically. The exceptions are exactly the features that require either a network or a secure origin, and they are worth stating precisely because a demonstration should not end in a surprising empty state.

Capability	Offline	Note
All public pages and routes	Works	Hash routing survives `file://`
Pass, muster, ledger, disputes	Works	Seeded data loads with the file
Exhibit mode and both portals	Works	Identical behaviour to the hosted build
QR display and printing	Works	Encodes the canonical public address
Camera scanner	Blocked by the browser	No secure origin on `file://`; manual entry offered
Read aloud	Depends on the device	Uses installed speech voices; hides itself if absent
Cross-device sharing	Not available	Records live in one browser profile

The camera limitation is not a defect to be hidden. When a judge asks why the scanner is greyed out, the honest answer — that browsers reserve camera access for secure origins, and a file opened from a USB stick is not one — is more convincing than an attempt to work around a deliberate browser protection.

CHAPTER 14 · IN ONE BREATH

- The offline edition packs the application, stylesheet, four typefaces and both photographs into one 1.9 MB file.
- Four defects — early execution, substitution corruption, spliced paths and a null origin — were invisible to source-level tests and only appeared when the file was opened.
- A twelve-check harness now opens that file from disk on every build, because this target failed silently four times.

CHAPTER 15

Testing and Validation

Four harnesses, 127 checks, and the defects they caught.

The project is verified by four independent harnesses rather than by manual clicking. Each harness answers a different question: does the arithmetic behave, does every screen survive rendering, does a real user journey complete, and does the offline file actually boot.

Harness	Checks	Question answered
Unit suite	75 assertions in 14 groups	Is the arithmetic, hashing, translation and role logic correct?
Render harness	9 screens	Does every public page produce content with no runtime error?
Flow test	31 steps	Can a user route, sign up, enter a portal, sign out and run the demo?
Offline harness	12 checks	Does the single file boot from disk with no network?

15.1 The fourteen unit groups

Group	Subject
1	Translation dictionary integrity and key parity
2	Parameterised translation and interpolation
3	Contact number and WhatsApp link correctness
4	Smart pass localisation and trade naming
5	Wage formulas and statutory overtime pro-rata
6	Demonstration-mode data protection
7	Responsive layout and horizontal overflow prevention
8	Authoritative breakdown arithmetic and de-duplication
9	Audit chain — all four verification outcomes
10	Anomaly rules RULE-01 to RULE-07
11	QR payload privacy and token safety
12	Role-based account lifecycle and scoping
13	Contact verification lifecycle and cross-checks
14	Real QR encoding, decoding and cross-device approval

15.2 The flow test, step by step

The flow test mounts the real application in a browser-like environment and drives it as a user would: it visits nine routes, audits four rendered screens for accessibility defects, switches the interface to Hindi and back, requests a route that does not exist, creates a worker account, enters the portal, checks four tabs, signs out, launches the exhibit mode, opens the pass, opens the welfare tab, opens the audit modal, then switches to the contractor view and checks the scanner and audit tabs. A failure at any step prints the step name and the DOM state that caused it.

The four accessibility audits inside it are worth describing, because they are the cheapest form of quality control in the project. For each audited screen the test collects every image without alt text, every button with no accessible name, every duplicated element id, and every unlabelled form field. The first run found four unlabelled fields and two icon-only buttons with no name, all of which were fixed rather than excluded.

15.3 Defects found and fixed

Defect	How it was found	Fix
Header controls remounted on every render	Flow test (focus lost)	Components hoisted to module scope
Pass QR pointed at a dead route	Scanning a printed card	New verification route; legacy payloads accepted
Same-day punch double-credited	Unit suite	Attendance write merges per worker and date
Cloud sign-in blocked local accounts	Manual test, offline	Error held; local lookup always performed
Welfare tab unreachable	Flow test	Tab added to the demo control
Modals without Escape or scroll lock	Accessibility audit	Shared modal behaviour hook
Hindi-first default for everyone	Flow test (language assertion)	Locale-driven default with persistence
Panic on a forbidden clipboard call	Manual test on HTTP	Guarded, with a visible fallback
Blank page on any render error	Offline harness	Application-wide error boundary

CHAPTER 15 · IN ONE BREATH

- Four harnesses answer four different questions: arithmetic, rendering, user journey and offline boot.
- 127 checks run together — 75 unit assertions, 9 rendered screens, 31 flow steps and 12 offline checks.
- Nine real defects were found and fixed by these harnesses, including a remount bug that silently destroyed focus.

CHAPTER 16

Results, Limitations and Honest Disclosures

What works, what does not, and what must not be claimed.

16.1 Results

At the time of writing, all four harnesses pass: 75 of 75 unit assertions, 9 of 9 rendered screens, 31 of 31 flow steps and 12 of 12 offline checks. The production build completes in under a second with a 612 kB main bundle, 182 kB gzipped, and role-scoped chunks that a public visitor never downloads. The offline edition is a single 1.9 MB file that mounts, renders and navigates with no network.

Demonstration scenario	Result
Sign up as a worker and land in a clean portal	Passes; empty state with no foreign data
Punch attendance and see it pending	Passes; duplicate punch merged
Approve the shift as a contractor	Passes; record enters the wage computation
Print the pass and scan it from another device	Passes; verification page resolves the token
Alter a stored ledger amount and re-verify	Passes; reports RECORD_MODIFIED at the exact event
Open the offline file from a USB stick	Passes; 12 of 12 checks

16.2 Limitations, stated plainly

1. **Storage is local.** Records live in one browser on one device. They are not synced, not backed up and not shared between a worker's phone and a contractor's laptop except through the demonstration data.
2. **Wage figures are reference values.** The minimum-wage floor is a prototype reference pending official notification. Every screen that uses it says so.
3. **No identity verification.** A scan proves a record exists; it does not prove the person holding the card is the worker named on it.
4. **Hashing is not immutability.** A local attacker with full device control can re-sign a rewritten chain. Detection requires an independent witness.
5. **Password hashing is fast.** A salted SHA-256 digest is better than plain text and weaker than Argon2id.
6. **Geofencing is modelled, not enforced.** The site reference exists on every attendance record, but the prototype does not check a GPS position against a fence.

7. **No offline sync.** The single-file edition runs entirely in memory of the tab; it does not reconcile with the hosted build.
8. **The camera needs a secure context.** On a plain-HTTP origin the scanner falls back to manual entry by design.

WHY THIS CHAPTER EXISTS

A project report that lists only successes is a sales document. The limitations above are the honest state of the work, and each one is also a concrete specification for the next version — which is exactly what Chapter 17 contains.

16.3 A five-minute demonstration

Exhibition time is short and judges arrive at unpredictable moments. The project was designed so that the most convincing part of it can be shown without preparation, in five minutes, in this order:

1. **Open the home page.** Let the pass card and the counted statistics land. Say one sentence about the problem, not about the technology.
2. **Enter the exhibit mode.** Show the worker's pass, then the wage ledger with its visible arithmetic.
3. **Scan the pass** with a second device, or open the verification link. Point out that the QR code contains no personal data.
4. **Break the chain.** Edit a stored amount in the browser's developer console, then press verify. The report names the exact event that no longer matches. This is the moment that convinces a technical judge.
5. **Switch to Hindi** and print the pass to A4. Then open the single-file edition with the network switched off.

Steps four and five are the ones worth rehearsing, because they are the only steps where something can visibly fail in front of an audience. Both have been run at least twenty times during preparation — the tamper demonstration because a single typo in the console produces a convincing-looking but meaningless result, and the offline build because it failed silently four separate times before it was trustworthy.

CHAPTER 16 · IN ONE BREATH

- Every harness passes and every demonstration scenario completes, including tamper detection at the exact altered event.
- Eight limitations are stated plainly, from local-only storage to the absence of identity verification.
- Limitation statements double as specifications for the next version rather than as apologies.

CHAPTER 17

Future Scope

What a real deployment would require, written as a specification rather than a wish list.

17.1 A server, and a conflict rule

The first requirement is a shared backend. Records would move to a hosted database with per-role row-level security, so that a contractor can read only the workers assigned to them and a worker can read only their own rows. The interesting problem is not storage but conflict: if a worker punches attendance offline and a contractor edits the same day online, the system needs a deterministic rule. The cleanest is append-only events with a last-writer-wins merge over an event log, keeping the loser as a visible revision rather than deleting it.

17.2 Independent witnesses

To make the audit chain genuinely tamper-evident rather than tamper-detectable, the chain head should be published somewhere the device does not control: a nightly digest posted to a public registry, a signed email to the worker, or an SMS containing the day's head hash. Any of these turns a local chain into an externally anchored one at almost no cost.

17.3 Institutional integration

- **e-Shram** for account-number validation, so a worker does not retype a number that can be checked.
- **BOCW** for registration status and welfare entitlement checks.
- **The district labour office** for a one-tap escalation that attaches the audit chain to a formal complaint — the single feature most likely to change an outcome in practice.
- **Bank or UPI rails** for payment acknowledgement, so that "paid" means a settlement reference exists rather than a checkbox.

17.4 Field hardening

- Argon2id password hashing with per-user salts, and a device-bound session.
- True geofenced punches with an accuracy radius and an explicit "location unavailable" state.
- Offline-first mobile capture with a durable local queue and conflict reporting on reconnection.
- A printed receipt for every settlement, because paper is still the medium a worker can take home.

- A usability study with twenty workers across two languages, which the current design has not yet had.

None of these is speculative technology. Each is a well-understood component; what they require is not invention but resources, institutional partnership and, above all, field time with the people the system is for.

If this project is continued, the next single step should not be more features. It should be twenty half-hour conversations with workers at three sites, recorded, transcribed and read back — because the design decisions in this report were derived from one such conversation and a great deal of inference. That inference is the weakest part of the work, and it is the part only field time can fix.

17.5 A plausible next-term plan

The items above are ordered by cost, not by ambition. If a single school term were available to continue the work, the sequence below would deliver a deployable pilot rather than a larger prototype.

Phase	Work	Artefact at the end
Weeks 1-2	Twenty recorded interviews across three sites	A decision log that corrects or confirms this report
Weeks 3-5	Hosted database with per-role row-level security	Two devices seeing the same record
Weeks 6-7	Offline capture queue and conflict reporting	A punch taken in a basement that reconciles on reconnection
Week 8	Chain head published nightly to an independent service	An externally anchored audit trail
Weeks 9-10	Argon2id passwords, device-bound sessions	A login that satisfies a security review
Week 11	Printed settlement receipt with a reference number	A paper artefact a worker can take home
Week 12	Usability test with twenty workers; rewrite copy from their words	A report chapter that is evidence rather than inference

CHAPTER 17 · IN ONE BREATH

- A shared backend with per-role row-level security and an append-only conflict rule is the first requirement.
- Publishing the chain head to an independent witness turns a local chain into an externally anchored one.
- Institutional integration — e-Shram, BOCW, the district labour office, payment rails — would change outcomes more than any interface work.

CHAPTER 18

Conclusion

A small, legible artefact in place of a large, unwritten assumption.

This project began with a question that could be asked in one sentence: can software give a daily-wage worker a record small enough to carry, private enough to show a stranger, and precise enough to argue with? After eighteen chapters the answer is a qualified yes. A worker can now hold a pass whose QR code reveals nothing about them, behind which sits a ledger whose arithmetic is visible line by line, protected by a chain that notices when its own history is altered, and presented in a language and at a size they can actually read.

The qualification matters as much as the yes. The record lives on one device. The wage floor is a reference, not a notification. A scan proves a record exists, not that its holder is who the card says. And a hash chain detects alteration without preventing it. Every one of those limits is printed in the product itself, not buried in an appendix, because a tool that overstates its guarantees is more dangerous than a tool that has none.

What was learned along the way is mostly about discipline rather than technology. The arithmetic had to be built and tested before a single screen existed, because a beautiful interface over wrong numbers is worse than nothing. The duplicate-punch rule had to live in the data layer rather than the interface, because an interface can always be bypassed. The offline build had to be tested by opening it, because four separate defects in it were invisible to every source-level test. And the hardest work in the whole project was not writing code but removing claims the system could not support.

For a student at **Mar Thoma H.S. School**, the most valuable outcome is not the application itself. It is the discovery that a careful record is a form of dignity: that when a worker can point at a line and say this day, this rate, this balance, the conversation stops being about who is believed and becomes about what is written down. Software cannot make an employer honest. It can make honesty legible, and legibility is where fairness starts.

The complete source, the offline edition and all four test harnesses accompany this report.

CHAPTER 18 · IN ONE BREATH

- The question was answered with a qualified yes: a portable, private, checkable record is achievable in software.
- The qualification is printed inside the product, not buried in an appendix.
- The most valuable outcome was the discovery that a careful record is a form of dignity — software cannot make an employer honest, but it can make honesty legible.

BIBLIOGRAPHY

References

[1] Government of India. Building and Other Construction Workers (Regulation of Employment and Conditions of Service) Act, 1976. Ministry of Labour and Employment.

[2] Government of India. Code on Wages, 2019. Ministry of Labour and Employment.

[3] Ministry of Labour and Employment. e-Shram — National Database of Unorganised Workers. https://eshram.gov.in

[4] Labour Department, Government of Madhya Pradesh. Minimum Wages Notification, Indore Region. Office of the District Labour Officer.

[5] NIST. FIPS 180-4: Secure Hash Standard (SHA-256). National Institute of Standards and Technology, 2015.

[6] W3C. Web Cryptography API. W3C Recommendation, 2017.

[7] W3C. Web Content Accessibility Guidelines (WCAG) 2.2. 2023.

[8] W3C. Web Application Manifest. W3C Working Draft.

[9] MDN Web Docs. IndexedDB, Service Worker API, Web Speech API, MediaDevices. Mozilla Foundation.

[10] React Documentation. Lazy loading, Suspense, Error Boundaries. Meta Open Source.

[11] Vite Documentation. Build options, library mode, single-file builds.

[12] World Bank. Social Protection for Informal Workers in South Asia. Discussion paper.

[13] International Labour Organization. Informal Economy and Decent Work. ILO reports.

[14] Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. (Referenced only for the hash-chaining idea.)

NOTE ON SOURCES

Statutory references were consulted for conceptual accuracy. The minimum-wage figures used in the application are prototype reference values and must not be cited as official rates.

APPENDIX A

Route Reference

Every addressable screen in the application, with the aliases the router accepts and the access rule that applies.

Route	Aliases	Access	Renders
`#/home`	—	Public	Landing page, pass sample, FAQ
`#/how-it-works`	`how`	Public	Six-step custody timeline
`#/about`	`about-project, bal-vigyan`	Public	Project background
`#/privacy`	`privacy-policy, data-use`	Public	Data commitments
`#/help`	`contact, support`	Public	Form, helpline, quick answers
`#/verify`	`verify-pass, pass`	Public	Token lookup and integrity report
`#/create-account`	`register, signup`	Public	Two-panel registration
`#/sign-in`	`login`	Public	Credential entry
`#/forgot-password`	`recover-password`	Public	Recovery request
`#/worker`	`worker-portal`	Worker session	Seven-tab portal
`#/contractor`	`contractor-portal`	Contractor session	Nine-tab dashboard
`#/judge-demo`	`demo`	Public (explicit)	Isolated exhibition suite
`not-found`	`any unknown`	Public	Real 404 with recovery routes

Deep-link grammar

Pass links take the form `#/verify?refToken=<token>`, where the token may be a worker identifier, an e-Shram number, a BOCW registration number or an opaque QR reference. The contractor scanner accepts the current shape and the legacy `#verify-<token>` shape, because a printed card cannot be recalled.

APPENDIX B

Test Index

The thirty-one steps of the flow harness and the twelve checks of the offline harness, as executed for this report.

1. App mounts on home
2. Route #/how-it-works
3. Route #/about
4. Route #/privacy
5. Route #/help
6. Route #/sign-in
7. Route #/create-account
8. Route #/forgot-password
9. Route #/verify
10. Accessibility audit · home
11. Language toggle switches public site to Hindi
12. Language toggle returns to English
13. Unknown route renders a 404 page
14. Home still reachable after 404
15. Verify route reports unknown reference
16. Accessibility audit · create account
17. Worker sign-up lands in portal
18. Accessibility audit · worker portal
19. New account starts clean
20. Portal shows approval tab
21. Worker wages tab renders
22. Worker profile tab renders
23. Sign out returns to public site
24. Judge demo launches
25. Judge demo has worker pass tab
26. Accessibility audit · judge demo
27. Judge demo welfare tab is reachable
28. Audit modal opens
29. Demo contractor view renders
30. Contractor scanner tab renders
31. Contractor audit tab renders

Offline harness

1. Single file exists (1.90 MB)
2. No external script tags
3. No external stylesheet links
4. No Google Fonts requests
5. Photographs embedded
6. Typefaces embedded
7. App mounted
8. Home screen rendered
9. Photographs render from embedded data
10. No uncaught runtime errors
11. Hash routing works offline
12. No remote subresources referenced by the DOM

Rendered screens

Home · HowItWorks · CreateAccount · SignIn · PasswordRecovery · Privacy · HelpContact · About · App shell (home route). Nine of nine rendered with the expected content and no runtime error.

APPENDIX C

Glossary

Term	Meaning
Muster roll	The daily register of who attended work on a site, and for how long.
e-Shram UAN	Universal Account Number issued by the e-Shram registry for unorganised workers.
BOCW	Building and Other Construction Workers Act, 1976 — the statute governing construction labour.
Paise	One hundredth of a rupee; the integer unit in which all money is computed.
Gross wages	Total earned from approved shifts before allowances and deductions.
Net due	Gross wages plus allowances minus itemised deductions.
Outstanding balance	Net due minus payments recorded; what is still owed.
Overpayment	The amount by which recorded payments exceed net due; never shown as a negative wage.
Advance (kharcha)	Money paid against future wages; reduces the balance.
Hash	A fixed-length fingerprint of data; the same input always produces the same output.
SHA-256	The standard secure hash used for every digest in the audit chain.
Genesis hash	The all-zero root value that the first audit event links to.
Chain linkage	The rule that each event's digest includes the previous event's digest.
Canonical serialisation	Deterministic ordering of an object's keys so that identical content hashes identically.
Anomaly rule	A deterministic check that raises a question about a record; it makes no accusation.
Opaque reference token	A meaningless identifier in a QR code that maps to a record only inside the system.
Skill category	Unskilled, semi-skilled, skilled or highly skilled; determines the statutory wage floor.
Half day	A shift paid at fifty percent of the daily base rate.
Pro-rata overtime	Overtime paid as base divided by eight, multiplied by extra hours.
Secure context	A browser state (HTTPS or localhost) required for camera and clipboard access.
Reduced motion	An operating-system preference that the interface honours by disabling animation.
localStorage	Browser storage used for records in this prototype, limited to one device.
Graceful degradation	Falling back to a usable alternative when a browser capability (camera, clipboard, storage, speech) is unavailable, rather than failing silently.

APPENDIX D

Selected Source Excerpts

Three short excerpts, reproduced from the working code, that carry the most weight in this report.

D.1 Integer paise conversion

src/services/wageEngine.ts

```
export function inrToPaise(inr: number): number {
  return Math.round(inr * 100);
}

export function paiseToInr(paise: number): number {
  return Math.round(paise) / 100;
}
```

D.2 Canonical serialisation

src/services/cryptoAudit.ts

```
const sortedKeys = Object.keys(data).sort();
for (const key of sortedKeys) {
  const val = data[key];
  if (val !== undefined) {
    entries.push(`${JSON.stringify(key)}:${canonicalSerialize(val)}`);
  }
}
return '{' + entries.join(',') + '}';
```

D.3 The balance rule

src/services/wageEngine.ts

```
if (paymentsRecordedPaise > netWagesDuePaise) {
  overpaymentPaise = paymentsRecordedPaise - netWagesDuePaise;
  outstandingBalancePaise = 0;
} else {
  outstandingBalancePaise = netWagesDuePaise - paymentsRecordedPaise;
}
```

These three fragments are the reason the rest of the system can be trusted with a number that a family depends on: conversion happens once, identical content hashes identically, and an overpayment can never be silently reported as a negative wage.

END OF REPORT

Mar Thoma H.S. School · Indore, Madhya Pradesh · Session 2026-27